

ASP: uma skill operacional de pre-mortem para agentes de código com LLM

Um relato de experiência

Carlos Ulisses Flores

Codex Hash Research Laboratory

ORCID: 0000-0002-6034-7765

Data: 7 de novembro de 2026

Resumo

Assistentes de código baseados em LLM muitas vezes produzem código *localmente correto* que omite modos de falha que um engenheiro sênior levantaria num pre-mortem — uma política de segurança por linha, o limite de taxa de uma API externa, o atraso de réplica durante uma migração —, e a produção então quebra de formas caras de depurar. Chamamos esses itens ausentes de *shadow points*. Este relato de experiência descreve o Anticipating Shadow Points (ASP), um protocolo de planejamento em 13 fases distribuído como skill do Claude Code, que integra o pre-mortem de Klein (2007) à taxonomia de falhas multiagente de Cemri et al. (2025), confere o plano com um validador que não compartilha o contexto do planejador, seguindo a literatura sobre os limites da autocorreção (Huang et al., 2024; Tyen et al., 2024), e entrega a execução a um subprocesso filho com contrato em arquivo. Documentamos o desenho, a implantação como plugin de código aberto e observações preliminares de cinco avaliações de engenharia, lidas pelo autor sem cegamento e sem transcrições preservadas. Um adjudicador determinístico acompanha a skill; suas notas sobre fixtures escritos pelo autor exercitam o instrumento e nada mais, por isso não relatamos efeito medido do ASP. Fixamos de antemão o resultado que mostraria um: num benchmark em modo real, com listas esperadas cegas e escritas por revisores independentes e oito réplicas por tarefa e condição, benefício medido se a cobertura média da condição assistida superar a da não assistida nas cinco tarefas e um teste de permutação unilateral, estratificado por tarefa, der $p < 0,05$; nenhum benefício mensurável se não a superar, empate incluído, em três ou mais; inconclusivo nos demais casos; e nenhum veredito se o orçamento interromper a coleta antes. O protocolo está redigido e ainda não foi depositado.

Palavras-chave: agentes de código LLM; pre-mortem; modos de falha multiagente; Claude Code; protocolos operacionais; relato de experiência

Whitepaper · Versão 2.1 · 2026-11 *(publicado pela primeira vez em 2026-05-18)*

DOI https://doi.org/10.5281/zenodo.20276631 *(DOI de conceito do repositório complementar · resolve para a sua versão depositada mais recente; esta versão do artigo é depositada com a release 2.0.0 e, até lá, o DOI resolve para a v1.0.3, que esta versão substitui)*

Status Whitepaper técnico independente · sem revisão por pares

Licença Texto, figuras e dados do artigo — CC BY 4.0 · Software complementar — MIT

Replicação Todo número que as Seções 5.3–5.4 calculam a partir dos dados e dos fixtures do repositório é recalculado por python run_all.py no repositório complementar e verificado por um teste; ver Apêndice B.

Repositório complementar Flores, C. U. (2026a), https://github.com/ulissesflores/anticipating-shadow-points

Whitepaper complementar Flores, C. U. (2026c). *Graceful Refusals as Unsignalled Failures: Characterising claude -p Exit-Code Semantics on Agent Declines.* Ver paper/graceful-refusals.md.

Abstract

LLM coding assistants often produce *locally correct* code that omits failure modes a senior engineer would surface in a pre-mortem — a row-level security policy, an external API's rate limit, replica lag during a migration — and production then breaks in ways that are expensive to debug. We call these missing items *shadow points*. This experience report describes Anticipating Shadow Points (ASP), a 13-phase planning protocol shipped as a Claude Code skill that integrates Klein's (2007) pre-mortem with the multi-agent failure taxonomy of Cemri et al. (2025), checks the plan with a validator that does not share the planner's context, following the self-correction-limit literature (Huang et al., 2024; Tyen et al., 2024), and hands execution to a child subprocess with a file-based contract. We document the design, its deployment as an open-source plugin, and preliminary observations from five engineering evaluations, read by the author without blinding and with transcripts not preserved. A deterministic adjudicator ships with the skill; its scores on author-written fixtures exercise the instrument and nothing more, so we report no measured effect of ASP. We state in advance the result that would show one: in a real-mode benchmark with blind, independently written expected lists and eight replicates per task and condition, a measured benefit if the assisted condition's mean coverage beats the unassisted one on all five tasks and a one-sided, task-stratified permutation test gives $p < 0.05$; no measurable benefit if it does not beat it, ties included, on three or more; inconclusive in between; and no verdict at all if the budget stops collection early. The protocol is drafted and not yet deposited.

Keywords: LLM coding agents; pre-mortem; multi-agent failure modes; Claude Code; operational protocols; experience report

1. Introdução

Equipes de software delegam cada vez mais trabalho de engenharia rotineiro — adicionar uma coluna a um banco de dados de produção, implantar uma função que chama uma API externa, refatorar um utilitário usado em trinta arquivos — a assistentes de código com IA. Quando o assistente acerta, o dia do engenheiro fica livre. Quando erra de forma *sutil* — escrevendo código que compila, passa nos próprios testes, vai para produção, mas omite em silêncio um passo que um engenheiro sênior teria apontado de antemão —, a produção quebra de formas caras de depurar. A ciência da decisão tem uma técnica para exatamente essa lacuna, o pre-mortem (Klein, 2007), e a pesquisa em agentes agora dispõe de um catálogo de como sistemas multiagente de LLM falham (Cemri et al., 2025). Nossa tese é que as duas pertencem ao mesmo lugar em tempo de execução: antes de escrever código, um agente deve ser levado a fazer um pre-mortem contra uma taxonomia de falhas fixa, e seu plano deve ser conferido por um validador que não enxerga o raciocínio do planejador.

O modo de falha dominante em produção dos agentes de código modernos com LLM, em nossa experiência, não é o código *ruim*. É o código *incompleto*: o agente faz o que foi pedido, mas não o que alguém com profundo contexto de domínio teria se perguntado *antes de começar*. O atraso de réplica durante uma migração de esquema. O cabeçalho CORS na edge function. A política de segurança por linha que precisa ser atualizada para acompanhar a coluna nova. Chamamos esses itens ausentes de **shadow points** — modos de falha óbvios para um profissional que já se queimou com eles, invisíveis para um agente que não se queimou.

Duas vertentes da literatura convergem nessa lacuna. A primeira, na ciência da decisão, é o **pre-mortem**: um exercício estruturado em que uma equipe imagina que um projeto já fracassou e enumera as causas antes de agir. A técnica foi operacionalizada para equipes de projeto por Klein

(2007) e fundamentada no trabalho empírico anterior de Mitchell, Russo e Pennington (1989) sobre retrospecção prospectiva (*prospective hindsight*); Klein (2007) a resume como um aumento de cerca de 30% na capacidade de identificar razões para um resultado futuro, número que não conseguimos recalcular a partir do artigo de 1989 (Apêndice A). A segunda, na pesquisa em agentes, é a caracterização recente dos modos de falha multiagente por Cemri et al. (2025), que derivaram uma taxonomia de 14 modos a partir de 150 traços iniciais de sistemas multiagente de LLM (concordância entre anotadores κ=0,88) e liberaram um corpus de mais de 1.600 traços anotados. Nenhuma das duas vertentes foi integrada como uma **disciplina de tempo de execução** que um agente de código possa ser levado a seguir em toda tarefa não trivial.

Este artigo relata o **Anticipating Shadow Points (ASP)**, um protocolo de planejamento em 13 fases distribuído como skill do Claude Code, que combina o raciocínio de pre-mortem com a lista de verificação de 14 modos do MAST, separa planejamento de validação sob estrito isolamento de prompt e entrega a execução a um subprocesso filho com contrato em arquivo. Implantamos a skill, distribuímo-la por três caminhos de instalação (marketplace de plugins, modo de desenvolvimento, script autônomo), submetemo-la a cinco avaliações de engenharia e relatamos o desenho e o que as avaliações podem e não podem mostrar. A avaliação é preliminar e, como corrigido nesta versão, não medida: GREEN em cinco tarefas e RED em três, uma execução cada, um único modelo, lidas pelo autor sem cegamento, com as transcrições brutas não preservadas. Relatamos isso como observação qualitativa; as quatro escolhas de projeto que tratamos como hipóteses, e uma quinta medida no artigo complementar, estão na Seção 6.

1.1. Contribuições

Este relato de experiência contribui com:

1. **O protocolo ASP como projeto documentado** — treze fases, cada uma emitindo um artefato nomeado, organizadas em torno do princípio de que nenhuma implementação começa antes de os shadow points terem sido levantados.
2. **Doze restrições operacionais a que chamamos de "Iron Laws"**, cada uma ligada a uma fonte primária da literatura ou a uma decisão de projeto documentada. Duas têm uma verificação em nível de shell no template do runner que o agente é instruído a executar (a saída JSON da Iron Law 12, um padrão do template que o operador pode sobrescrever, e a flag ASP_IN_GOAL da Iron Law 9, definida para o filho, cuja verificação na entrada é um trecho de Bash que a skill manda o agente executar); as outras dez são instruções em nível de prompt.
3. **Observações preliminares e qualitativas** de cinco avaliações de engenharia, e um adjudicador determinístico com proveniência criptográfica, exercitado em fixtures construídos e pronto para pontuar transcrições reais sem confiar na adjudicação do autor (as listas esperadas continuam sendo do autor, e o sidecar opcional de sinônimos não é fornecido para nenhuma tarefa).
4. **Um achado empírico separado** sobre a semântica do código de saída do claude -p sob recusa educada do agente, caracterizado no artigo complementar Flores (2026c). O achado motivou duas das Iron Laws, mas merece caracterização independente com N maior.

Este é um **relato de experiência** no sentido de sistemas, não um artigo de benchmark. Documentamos um sistema que construímos, uma metodologia que implantamos e o que observamos; não alegamos uma contribuição empírica robusta à literatura de benchmarks de agentes de LLM. A Seção 7 enumera em detalhe as ameaças à validade.

2. Fundamentação

Revisamos quatro vertentes de trabalhos recentes que informam diretamente o desenho do ASP (§§2.1–2.4) e depois situamos o ASP entre as disciplinas mais antigas de engenharia de software e de qualidade que ele transpõe (§2.5). Mantemos a revisão dos trabalhos recentes curta em relação ao habitual nos artigos de sistemas de agentes, porque a contribuição aqui é operacional e o contexto da literatura está a serviço dela.

2.1. Pre-mortem e retrospecção prospectiva

A técnica do pre-mortem pede a uma equipe que imagine, antes do compromisso, que um projeto já fracassou de forma catastrófica, e que enumere as causas. A base empírica é Mitchell, Russo e Pennington (1989), que mostraram que imaginar um evento futuro como certo (e não como incerto) produz explicações mais longas e mais episódicas; Klein (2007) resume o efeito como um ganho de cerca de 30% na identificação correta de razões para um resultado futuro. Klein (2007) operacionalizou o achado como o “pre-mortem” de equipes de projeto, e a técnica se espalhou desde então pela ciência da decisão e pela prática de gestão de projetos. Sua base experimental é tênue: não encontramos replicação nem metanálise do efeito do pre-mortem no Crossref, no arXiv nem numa busca geral na web (novembro de 2026), o que é uma ausência nesses índices, não prova de que nenhuma exista. Normas de avaliação de risco catalogam técnicas vizinhas — a análise de cenários e a técnica estruturada do “e se” (SWIFT) —, mas o sumário da IEC

31010:2019, Anexo B, não lista o pre-mortem pelo nome. O termo *prospective hindsight* também foi reaproveitado para um sinal de aprendizado por reforço sem relação com ele (J. Zhang et al., 2026); nós o usamos apenas no sentido de Mitchell et al. Não encontramos integração anterior do pre-mortem como disciplina de tempo de execução para agentes de código.

2.2. Modos de falha multiagente (MAST)

Agentes de LLM intercalam raciocínio e ação num único ciclo (Yao et al., 2023) e são cada vez mais compostos em fluxos de trabalho multiagente cujos modos de falha se acumulam. Cemri et al. (2025) introduzem a **Multi-Agent System Failure Taxonomy (MAST)** com o **MAST-Data**, um corpus de mais de 1.600 traços anotados de sete frameworks multiagente populares, como o AG2 (antigo AutoGen; Wu et al., 2023). A partir de uma análise rigorosa de 150 traços iniciais (concordância entre anotadores κ=0,88), derivaram quatorze modos de falha organizados em três categorias: especificação e projeto do sistema (5 modos), desalinhamento entre agentes (6 modos) e verificação e término de tarefas (3 modos). A lista de 14 modos é a fundamentação empírica recente mais forte para uma lista de verificação de modos de falha; o ASP adota os nomes dos modos literalmente e operacionaliza cada um num prompt de pre-mortem mais uma mitigação, nos arquivos da skill complementar. Dois trabalhos concorrentes estendem a direção da taxonomia: Zhu et al. (2025) introduzem o AgentDebug e a AgentErrorTaxonomy (memória / reflexão / planejamento / ação / nível de sistema), e S. Zhang et al. (2025), um Spotlight do ICML 2025, abre a **atribuição automatizada de falhas** com o conjunto de dados Who&When. O MAST é, portanto, uma taxonomia entre várias: o TRAIL formaliza uma taxonomia de erros agênticos sobre 148 traços anotados por humanos (Deshpande et al., 2025), o AgentRx anota 170 trajetórias fracassadas com um passo crítico e uma categoria de uma taxonomia de teoria fundamentada (Barke et al., 2026), e o AgenTracer relata que os LLMs de raciocínio atuais atribuem falhas em traços

multiagente com acurácia geralmente abaixo de 10% (G. Zhang et al., 2025). O ASP usa o MAST como uma biblioteca de prompts para antecipar falhas, não como classificador das observadas, de modo que a escolha da taxonomia delimita o que a Fase 2 pergunta, e não o que ela consegue detectar.

2.3. Limites da autocorreção e isolamento do validador

O argumento isolado mais forte para separar o validador do planejador é o achado convergente, em três trabalhos primários, de que os LLMs não conseguem criticar a si mesmos de forma confiável sob contexto compartilhado. Huang et al. (2024, ICLR) mostram que a autocorreção intrínseca, sem retroalimentação externa, muitas vezes degrada o desempenho. Tyen et al. (2024) decompõem a autocorreção em localização e correção e mostram empiricamente que os LLMs não conseguem *localizar* os próprios erros de raciocínio, mas conseguem corrigi-los quando a localização é dada externamente — *a localização é o gargalo*. Zheng et al. (2023, NeurIPS D&B) documentam que juízes LLM fortes favorecem em excesso as saídas de modelos da mesma família (um "viés de autoaprimoramento"), mesmo ao julgar objetivamente a qualidade. Kamoi et al. (2024) fazem um levantamento dessa literatura e concluem que a autocorreção confiável de erros de raciocínio exige retroalimentação externa ou verificadores, e não o julgamento do modelo sem auxílio; métodos de autorretroalimentação intrínseca como o Self-Refine (Madaan et al., 2023) melhoram algumas tarefas de geração, mas não fecham a lacuna de localização que Tyen et al. (2024) isolam. Juntos, esses achados sustentam que um validador que compartilha contexto com um planejador herda a calibração de confiança do planejador sobre o conteúdo compartilhado, inclusive sobre o conteúdo errado de que o planejador está confiante. O desenho do ASP impõe estrito isolamento de prompt entre planejador e validador (Iron Law 2, adiante) como contramedida arquitetural. Kambhampati et al. (2024) chegam à mesma conclusão para o

planejamento: LLMs autorregressivos não conseguem, por si sós, planejar nem autoverificar, mas podem servir como fontes aproximadas de conhecimento dentro de um ciclo “LLM-Modulo” com verificadores externos — o papel que o ASP atribui ao validador isolado e aos testes de aceitação executáveis.

2.4. Trabalho relacionado mais próximo: o PreFlect

O trabalho concorrente mais próximo é o **PreFlect** (Wang et al., 2026), que de modo semelhante desloca o paradigma da autorreflexão da correção a posteriori para a crítica de planos de agentes antes da execução, destilando erros de planejamento a partir de trajetórias históricas de agentes e acrescentando um mecanismo dinâmico de replanejamento. A semelhança de alto nível é significativa: tanto o ASP quanto o PreFlect identificam como limitação o caráter exclusivamente pós-execução da reflexão convencional e deslocam a crítica para *antes* da ação. O ASP difere do PreFlect em quatro eixos:

1. **Validador isolado por prompt.** O PreFlect destila erros na própria crítica pré-execução do agente; o ASP impõe um subagente validador de prompt separado, que nunca vê a transcrição do planejador.
2. **Isolamento em nível de processo para o executor.** O ASP cria o executor como um subprocesso claude -p com contrato em arquivo; o PreFlect opera dentro de um único processo.
3. **Restrições explícitas de tempo de execução.** O ASP enuncia doze Iron Laws, duas delas protegidas por verificações executáveis e dez como regras em nível de prompt; o PreFlect é apresentado como uma metodologia, e não como um sistema de restrições.

4. **Um achado empírico no nível da CLI** que caracteriza a semântica de saída do claude -p sob recusa educada, motivando duas das Iron Laws; ele é relatado separadamente em Flores (2026c). O fenômeno em si é documentado pelo fornecedor e em relatos públicos de issues (catalogados em Flores, 2026c, §2); a contribuição ali é uma contagem medida e uma proposta de contrato de término tipado, não uma descoberta.

2.5. Linhagem clássica de engenharia de software e de qualidade

As quatro vertentes acima situam o ASP na literatura recente de agentes de LLM. Um revisor de engenharia de software ou de qualidade reconhecerá, porém, a maquinaria do ASP como uma transposição de disciplinas que antecedem os LLMs em décadas. Tornamos o empréstimo explícito, porque escondê-lo seria desonesto e mais fraco: o que é novo no ASP não são as disciplinas, mas a sua transposição para um ambiente de execução de agentes não determinístico e os mecanismos que as impõem ali.

Não redundância versus reverificação defensiva: onde o ASP se afasta do Design by Contract. O esquema de micro-TODO do ASP (§3.4) é o Design by Contract (Meyer, 1992) transferido da fronteira da rotina para a fronteira da tarefa do agente. O contrato de Meyer distribui obrigações entre uma fronteira chamador/rotina: o chamador deve estabelecer a PRECONDIÇÃO, que a rotina tem então o direito de *presumir* — o princípio de não redundância de Meyer proíbe o corpo de reverificá-la — e a rotina deve garantir a PÓS-CONDIÇÃO em que o chamador se apoia. Dos sete campos do esquema, PRECONDITION, ACTION e POSTCONDITION mapeiam diretamente para esse contrato, com ACTION como o corpo da rotina que o par pré/pós envolve. O ASP então *se afasta* de Meyer de propósito: como o obrigado é um modelo não determinístico, e não uma rotina compilada, o ASP não presume a pós-condição — ele a reverifica com um ACCEPTANCE-TEST externo — e acrescenta um FALSIFICATION-TEST que não tem análogo

na programação por contrato, importando em seu lugar a postura popperiana (Popper, 1959/2002) de que uma afirmação ganha confiança por sobreviver a uma tentativa de refutação, e não apenas por uma verificação confirmatória. Essa troca da não redundância do Design by Contract pela reverificação defensiva é o mesmo ethos de não confiar em nada que produz a disciplina de código de saída das Iron Laws 11–12; é uma inversão ponderada de Meyer, não um descuido. As Iron Laws, por sua vez, são enunciadas como o análogo do terceiro elemento do contrato do DbC — a invariante de classe — e não como uma condição pré/pós por tarefa; dez delas são instruções que o agente deve manter, não verificações que rodam.

Enumeração de modos de falha. A detecção de shadow points (Fase 2) é uma enumeração prospectiva de modos de falha na tradição da FMEA (MIL-STD-1629A, 1980; IEC 60812:2018). Ela deliberadamente *não* é uma FMEA completa: o ASP não faz decomposição por componente nem pontuação de prioridade de risco (severidade × ocorrência × detecção), de modo que a dívida é com a disciplina de enumeração prospectiva da FMEA, não com a sua mecânica de baixo para cima. Três insumos da Fase 2 ocupam papéis que vale separar: o pre-mortem (Klein, 2007; §2.1) fornece a *filosofia* — imaginar a falha como já certa; a taxonomia de catorze modos do MAST (Cemri et al., 2025; §2.2) fornece a *biblioteca* de modos de falha; e a enumeração forçada contra essa biblioteca fixa se parece, estruturalmente, mais com o uso de palavras-guia no estilo HAZOP (IEC 61882:2016) do que com a FMEA componente a componente. O que o ASP acrescenta é o substrato — o "componente" cujos modos de falha são enumerados é um plano de LLM, e a enumeração é feita por um validador que não compartilha o contexto do planejador (§2.3).

À prova de erro (mistake-proofing). Um subconjunto das Iron Laws (§3.3) são poka-yoke (Shingo, 1986) em nível de protocolo: o template do runner em claude-p-goal-runner.md define ASP_IN_GOAL para o filho e sempre passa --output-format json (Iron Laws 9 e 12); o contrato

em arquivo da Iron Law 10 é uma instrução. Essas verificações vivem num template Bash que o agente é instruído a executar, e não num hook fora do controle do agente, e não são a impossibilidade física do poka-yoke verdadeiro — uma flag de ambiente pode ser removida ou deixar de se propagar —, e as demais leis são honestamente portões procedimentais (Iron Laws 1, 2, 4, 5) ou regras de verificação que ainda dependem do operador, das quais a Iron Law 11 ("nunca confiar em $?") é o caso mais claro: precisamente a dependência da vigilância do operador contra a qual o poka-yoke é definido. O empréstimo é a *postura* de prova de erro aplicada a duas verificações de shell num script de runner que o agente é instruído a emitir, não uma alegação de que o ASP elimina a vigilância do operador.

Verificação e validação independentes. O validador do ASP isolado por prompt (Iron Law 2) transpõe o movimento definidor da Verificação e Validação Independentes (IV&V; IEEE Std 1012-2016) — um verificador mantido independente do desenvolvedor, nos termos da IEEE 1012 técnica, gerencial e financeiramente — da separação organizacional para a separação de *prompt*: o subagente validador não compartilha contexto com o planejador que confere. A justificativa específica de LLM para esse isolamento é a literatura de autocorreção da §2.3; a *forma* da contramedida é a da IV&V.

Vocabulário de dependabilidade. O comportamento do código de saída por trás das Iron Laws 11–12 tem um nome exato na taxonomia de dependabilidade: um exit 0 numa recusa educada é uma *falha não sinalizada* (Avizienis et al., 2004), nem uma pane nem uma parada fail-silent. O termo, sua literatura anterior e a medição são desenvolvidos no artigo complementar (Flores, 2026c); esta seção importa apenas o vocabulário e não reabre aqui a questão da novidade.

Lastro empírico. A preferência por testes de aceitação e de falsificação em vez de códigos de status em que se confia tem base empírica na patologia adjacente. Yuan et al. (2014) constataram

que a grande maioria das falhas catastróficas em sistemas distribuídos intensivos em dados remonta ao tratamento defeituoso de erros não fatais que *foram* sinalizados. O caso do código de saída do ASP é o complemento — o sinal estava *ausente* —, de modo que os dois não compartilham uma tese; a lição estreita que compartilham é que um canal de status não é um proxy do desfecho, e é isso que um ACCEPTANCE-TEST externo existe para impor.

Tabela 1 mapeia cada mecanismo do ASP para o seu ancestral clássico e declara o que o ASP acrescenta.

Tabela 1
Mecanismos do ASP e seus ancestrais clássicos

Mecanismo do ASP	**Ancestral clássico**	**O que o ASP acrescenta (e *não* alega)**
micro-TODO PRE/ACTION/POST + ACCEPTANCE-TEST	Design by Contract (Meyer, 1992)	um obrigado não determinístico cuja pós-condição é reverificada por comando externo, e não presumida; não a ideia do contrato
micro-TODO FALSIFICATION-TEST	Falsificação popperiana (Popper, 1959/2002)	uma etapa de refutação na fronteira da tarefa do agente; sem análogo no Design by Contract
Detecção de shadow points (Fase 2)	Enumeração no estilo FMEA (MIL-STD-1629A; IEC 60812:2018) + palavras-guia HAZOP, sobre a biblioteca MAST (Cemri et al., 2025)	enumeração de um plano de LLM sob isolamento; sem pontuação de prioridade de risco, sem decomposição por componente
Iron Laws 9 e 12 como verificações de shell	Poka-yoke (Shingo, 1986)	duas verificações num script de runner que o agente é instruído a emitir; não é impossibilidade física, e a Iron Law 10 é uma instrução
Validador isolado por prompt (Iron Law 2)	IV&V (IEEE Std 1012-2016)	independência organizacional reformulada como isolamento de prompt; não é IV&V
Nunca confiar em $? (Iron Laws 11–12)	Falha não sinalizada (Avizienis et al., 2004); Yuan et al. (2014)	caracterizada no artigo complementar (Flores, 2026c); não é a classe de falha

Lida em conjunto, a seção faz uma alegação deliberadamente modesta: a maquinaria do ASP é uma *integração* de disciplinas estabelecidas de confiabilidade e de engenharia de qualidade — contrato, enumeração de modos de falha, IV&V, prova de erro — num único protocolo cuja

contribuição é operacional, e não conceitual. O que essa integração acrescenta além dos seus ancestrais é a sua aplicação num ambiente de execução de agentes não determinístico: as Iron Laws (duas delas protegidas por verificações executáveis), o isolamento em nível de prompt e de processo da §3.2 e o adjudicador determinístico reprodutível da §5, com o achado do código de saída do claude -p caracterizado separadamente no artigo complementar. Essa é a alegação de relato de experiência da §1, não uma alegação de novidade contra a literatura clássica.

3. O Protocolo ASP

3.1. Visão geral

O ASP é um protocolo de 13 fases que progride da admissão (intake) até a aprovação da entrega. Cada fase emite um artefato nomeado (Tabela 2). O princípio que organiza as fases é que nenhuma implementação começa antes de os shadow points terem sido levantados (Fase 2), validados de forma independente (Fase 5) e aprovados pelo usuário (Fase 7); a execução é delegada a um subprocesso filho com contrato em arquivo (Fase 9).

Tabela 2

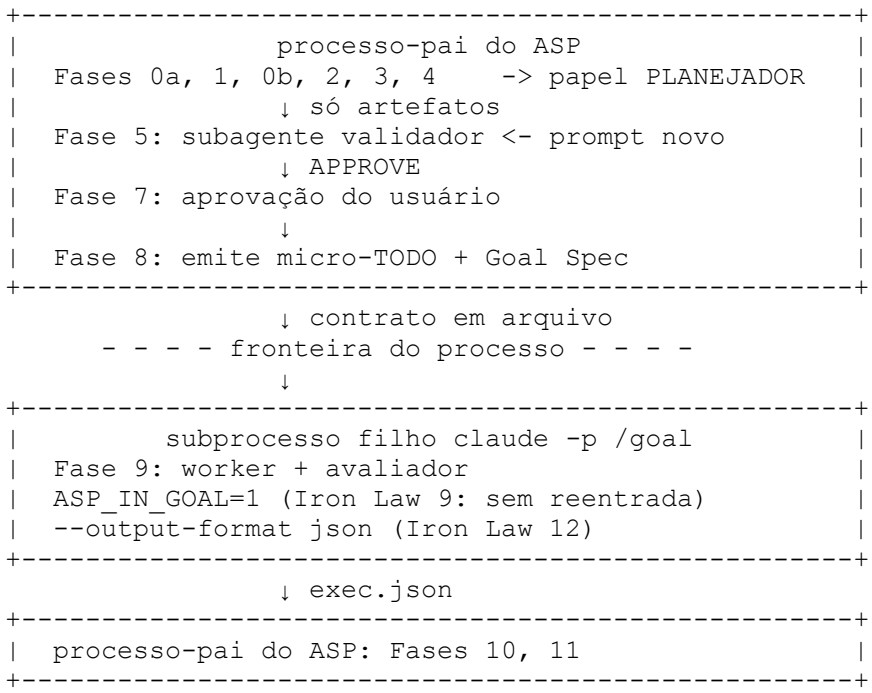
Fases do ASP e seus artefatos emitidos. A Fase 0 tem duas subfases (0a antes da pesquisa, 0b depois dela), de modo que treze fases ocupam quatorze linhas

#	Fase	Artefato	Justificativa de projeto
0a	Perguntas e respostas pré-pesquisa	bloco intake	Boa prática do Anthropic Skills: saídas intermediárias verificáveis
1	Pesquisa paralela	Resumo da pesquisa + bloco de recall	Padrão de despacho de subagentes
0b	Perguntas e respostas pós-pesquisa	Intake refinado (dispensável)	Igual à 0a
2	Detecção de shadow points	Lista categorizada + mitigações	Klein (2007) + Cemri et al. (2025)
3	Project Charter	Charter preenchido	Passagem durável baseada em artefatos
4	Plano macro + registro	Plano macro + entregáveis D01–D*n*	Plan-and-Act (Erdogan et al., 2025)
5	Validador independente	APPROVE / REVISE	Huang et al. (2024) + Tyen et al. (2024) + Zheng et al. (2023)
6	Ciclo interno (limite=3)	Plano da revisão final	Iteração limitada; FM-1.5
7	Aprovação do usuário	Go / no-go	Portão com humano no ciclo
8	Micro-TODO + Goal Spec	TaskCreate por passo + Goal Spec	Passagem ao executor
9	Execução via subprocesso /goal	Tarefas marcadas como concluídas com evidência	Isolamento em nível de processo; ver Flores (2026c)
10	Aprovação por entregável	Cada entregável aceito ou rejeitado	Contrato de aceitação
11	Relatório de execução + memória	Artefato de encerramento + gravação na memória	Memória no estilo Reflexion (Shinn et al., 2023)
12	Publicação pública (opcional)	Marketplace de plugins + release no GitHub	Camada de distribuição

3.2. Posicionamento arquitetural

O ASP separa três papéis de agente com passagens explícitas apenas por artefatos (Figura 1):

Figura 1
Separação em três papéis do ASP. As fases do planejador rodam no processo pai



Nota. O validador roda no pai, mas num prompt novo de subagente que recebe apenas artefatos, nunca a transcrição do planejador. O worker roda num subprocesso filho claude -p com contrato em arquivo.

3.3. Iron Laws

Enunciamos doze restrições operacionais e as chamamos de Iron Laws (Tabela 3). As Iron Laws 9 e 12 têm verificações em nível de shell no template do runner e no trecho de entrada que o agente é instruído a executar; as demais são regras que a skill instrui o agente a seguir. O cumprimento de qualquer dos dois tipos não foi medido. Cada uma é mapeada à sua derivação: uma fonte

primária da literatura, uma recomendação documentada da Anthropic ou uma decisão empírica de projeto.

Tabela 3
As 12 Iron Laws e sua derivação

#	Lei	Derivação
1	Nenhuma implementação sem antes passar pelos shadow points	Klein (2007) + Cemri et al. (2025)
2	Nenhuma aprovação de plano sem passagem por validador independente (prompt separado)	Huang et al. (2024) + Tyen et al. (2024) + Zheng et al. (2023); mitiga FM-3.3
3	Nenhum micro-TODO sem plano macro + charter + entregáveis aprovados pelo usuário	Anthropic Skills: saídas intermediárias verificáveis
4	Nenhum passo concluído sem evidência recente de teste de aceitação	Cemri et al. (2025) FM-3.1 + FM-3.2 + FM-2.6
5	Nenhum entregável aceito sem correspondência com arquivo/saída de evidência	Cemri et al. (2025) FM-3.1
6	Nenhum encerramento de projeto sem relatório de execução + gravação na memória	Shinn et al. (2023), buffer de memória
7	Nenhum ciclo sem limite (limite padrão = 3 rodadas de validador)	Cemri et al. (2025) FM-1.5
8	Nenhuma execução da Fase 9 sem despacho de avaliador nos pontos de verificação (exceto sob --quick)	Erdogan et al. (2025), planejador/executor
9	Nenhum ASP reentrante dentro de um filho claude -p /goal (guarda por variável de ambiente ASP_IN_GOAL)	Mitigação de explosão de custo; decisão de projeto
10	Nenhuma criação de claude -p /goal sem contrato em arquivo	Mitigação de divergência de estado do subprocesso; decisão de projeto
11	Nunca confiar em $? do claude -p para sucesso/falha semântica	Observação empírica, caracterizada em Flores (2026c)
12	Toda invocação de claude -p deve usar --output-format json ou stream-json	Corolário da Iron Law 11

3.4. O contrato de micro-TODO

Na fronteira entre planejamento e execução (Fase 8), o ASP emite uma lista de tarefas contratual. Cada tarefa é criada por meio da primitiva TaskCreate com sete campos rotulados:

- **PRECONDITION** — invariante que deve valer antes do passo.
- **ACTION** — comando exato ou edição literal.
- **POSTCONDITION** — mudança de estado efetuada pelo passo.

- **ACCEPTANCE-TEST** — comando e saída esperada que provam a pós-condição.
- **FALSIFICATION-TEST** — comando ou observação que provaria que o passo falhou.
- **DEPENDS-ON** — identificadores das tarefas a montante (estrutura de DAG).
- **DELIVERABLE-ID** — a qual entregável do registro este passo contribui.

Esse esquema é o que torna a lista de tarefas auditável: nenhuma tarefa é marcada como concluída sem evidência de teste de aceitação (Iron Law 4); nenhum entregável é aprovado sem evidência correspondente (Iron Law 5). O esquema é compartilhado com o Goal Spec consumido pelo subprocesso da Fase 9.

4. Implantação Operacional

O ASP é distribuído como um plugin do Claude Code, por três caminhos de instalação, cada um adequado a um público diferente:

1. **Marketplace de plugins** (recomendado para usuários finais):

   ```
   claude plugin marketplace add ulissesflores/anticipating-shadow-points
   claude plugin install anticipating-shadow-points@anticipating-shadow-points
   ```

   Invocação: /anticipating-shadow-points:asp <task> ou apenas /asp.

2. **Modo de desenvolvimento** (para contribuidores): claude --plugin-dir ./ carrega a skill no próprio lugar; útil ao iterar sobre a skill em si.

3. **Script de instalação autônomo** (para quem não usa o gerenciador de plugins): ./scripts/install.sh copia a skill para o diretório de skills do Claude Code do usuário.

O plugin usa o formato oficial de manifesto .claude-plugin/plugin.json da Anthropic (Claude Code 2.1+; Anthropic, 2026a), e o repositório funciona também como marketplace próprio por meio de .claude-plugin/marketplace.json na raiz do repositório.

Permissões. Desde a versão 1.0.4 do plugin, o runner da Fase 9 cria o filho claude -p **sem nenhuma flag de permissão**: o filho roda sob as configurações de permissão do próprio usuário no Claude Code, e a skill nunca as concede, amplia ou contorna. As versões anteriores passavam --permission-mode bypassPermissions; isso foi removido depois que o diretório de plugins do Claude recusou a listagem, porque um plugin não pode conceder permissões a si mesmo. No modo não interativo, uma ação que o usuário não permitiu previamente é negada, e não perguntada; o pai lê o campo permission_denials do resultado JSON do filho e relata ao usuário qualquer negação, em vez de tentar de novo com permissão mais ampla. Isso torna a execução autônoma estritamente mais fraca do que na implantação de maio de 2026 e mais segura: a skill só pode fazer o que o operador já permitiu. A própria skill é implementada como uma skill em Markdown seguindo o documento de boas práticas de Agent Skills da Anthropic (Anthropic, 2026b), com arquivos complementares para a lista de verificação do MAST, os prompts de pre-mortem por modo e os sete templates de artefato referenciados na Seção 3.1.

A documentação é publicada como um site no GitHub Pages, com traduções do README para português, espanhol, italiano e hebraico, todas feitas com assistência de máquina ou de IA: a portuguesa aguarda a revisão do autor, as outras três a de um falante nativo (docs/translations.md). Todo o código-fonte — arquivos da skill, manifestos do plugin, documentação, artefatos de teste, código do benchmark — é licenciado sob MIT; o texto do artigo, as figuras e os dados são CC BY 4.0.

5. Observações Preliminares

Submetemos o ASP a cinco avaliações estruturadas de engenharia, que abrangem domínios heterogêneos. Esta seção separa três coisas que a versão 1.0 misturava: o que o autor observou em

execuções reais de modelos (5.2), o que o adjudicador determinístico mostra sobre si mesmo em fixtures construídos (5.3) e o que não foi medido de forma alguma.

5.1. O conjunto de avaliações

Tabela 4
As cinco avaliações de engenharia

ID	Domínio	Tarefa (abreviada)	Tamanho esperado	Aceitação
E1	Migração de esquema	Coluna NOT NULL adicionada a user_profiles com 1M de linhas	10	≥ 80%
E2	Deploy em edge	Edge function que chama uma API externa com limite de taxa	10	≥ 80%
E3	Refatoração	Utilitário de formatação de datas usado em mais de 30 arquivos	10	≥ 80%
E4	Política RLS	Política de segurança por linha que referencia uma coluna nova	10	≥ 80%
E5	Agendamento	Job cron cujo script conflita com um serviço desativado	10	≥ 80% e pontos 1 e 8 presentes

Cada avaliação especifica um INPUT (um enunciado de tarefa como um engenheiro sênior o formularia a um colega), uma lista EXPECTED de shadow points que um profissional competente levantaria e um limiar de ACCEPTANCE (padrão de 80%). A E5 não é portável: dois de seus dez pontos esperados (1 e 8) dependem da memória privada de projeto do autor e não podem surgir apenas a partir do seu INPUT, de modo que ela mede a recuperação de memória tanto quanto o planejamento. O texto completo de cada avaliação está no repositório complementar.

5.2. O que foi observado em execuções reais (autor, não cego)

Em 2026-05-17, as avaliações foram despachadas a subagentes novos: GREEN (com a disciplina do ASP injetada: enquadramento de pre-mortem, enumeração forçada dos 14 modos do MAST, exigência de mitigação) nas cinco, e RED (instruído a planejar como um engenheiro sênior faria, sem nenhum framework) em três, E1–E3; a E4 e a E5 foram acrescentadas depois e executadas apenas em GREEN. Todas rodaram no modelo da sessão da época (Claude Opus 4.7; o modelo

não foi registrado por execução), uma execução por condição. Os subagentes herdaram as instruções globais de agente do autor, de modo que a condição RED era uma linha de base competente, e não ingênua. O autor leu cada plano contra a lista esperada e registrou um juízo de cobertura; os resumos, com os identificadores dos subagentes, estão versionados em tests/baseline-pressure-tests.md e tests/evals-summary.md. As transcrições brutas não foram versionadas e já não estão disponíveis. Os prompts RED registrados ali são formulações em português, mais curtas, de E1–E3, que omitem detalhes que as listas esperadas depois pontuam: o prompt RED da E2 não declara o limite de 100 req/s do Resend, e o da E1 não diz que o código da aplicação já referencia a coluna nova em cinco pontos de consulta. Parte da deficiência de RED pode, portanto, ser um artefato do prompt, e não da ausência do protocolo.

Sob essa leitura não cega, os três planos RED cobriram cerca de 30–60% de suas listas esperadas (média perto de 47%, contando menções parciais como cobertas) e os planos GREEN cobriram todas ou quase todas as suas. Os resumos versionados, escritos em maio de 2026, concluem que o ASP é “reprodutivelmente melhor” (tradução nossa); uma nota acrescentada acima deles na versão 2.0 retira essa conclusão, que não é sustentada. Essas cifras são um juízo do autor sobre texto não preservado, pontuado contra listas escritas pelo mesmo autor, com o vocabulário do MAST que o prompt GREEN injeta aparecendo nessas listas. São relatadas como observação, não como medição, e nenhum número deste parágrafo é verificado pela camada de replicação.

5.3. O que o adjudicador determinístico mostra sobre si mesmo

O pacote benchmark/ pontua um plano contra uma lista esperada com um matcher determinístico de 4 camadas (nome exato, sidecar de sinônimos, janela deslizante de Jaccard, sem correspondência). Na execução publicada, todas as 42 correspondências vêm da primeira camada (nome exato); não existe sidecar de sinônimos para nenhuma tarefa, e a camada de Jaccard nunca

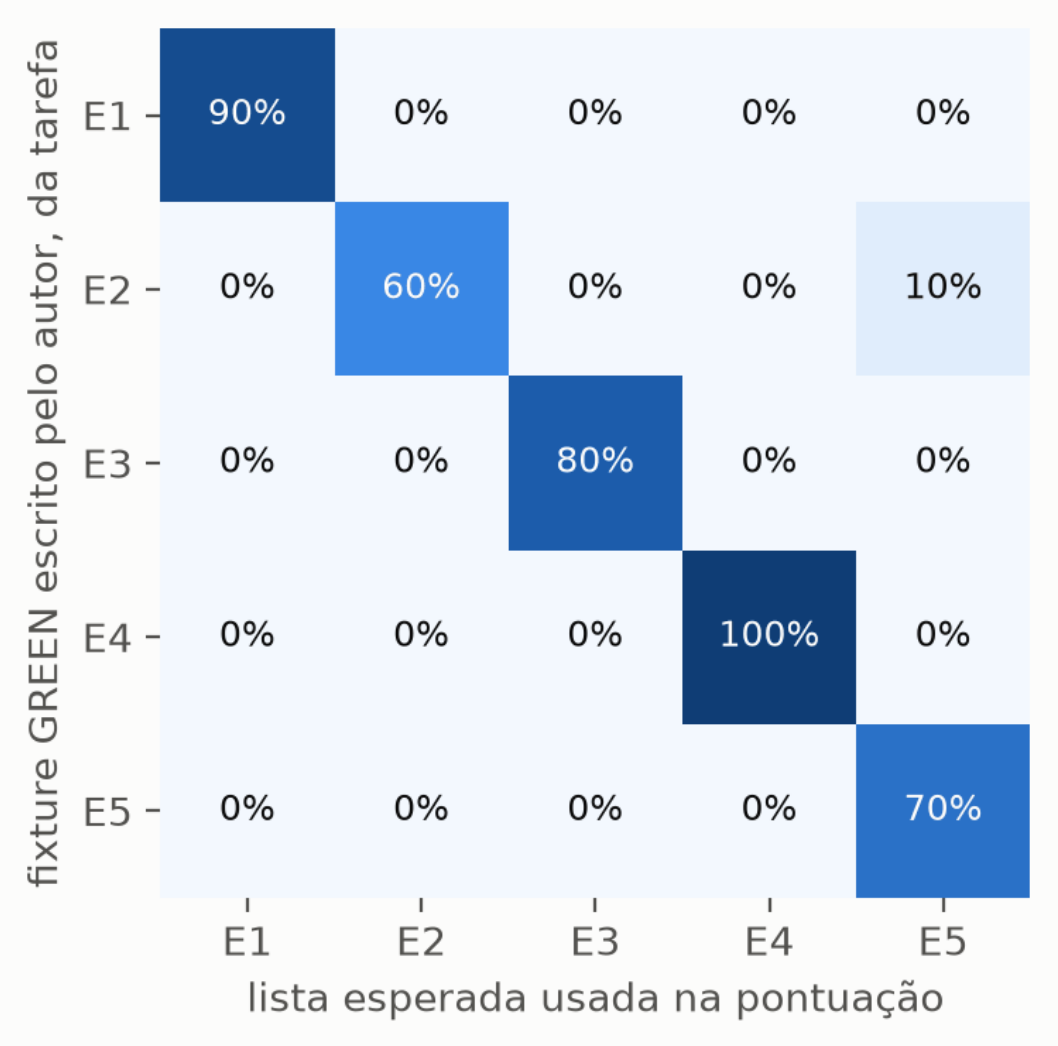
dispara. Sua única execução publicada é uma execução em **modo mock**: os textos RED e GREEN são fixtures que o autor escreveu para exercitar o pipeline de ponta a ponta (benchmark/tests/fixtures/mock_runs/); neles, as camadas 2 e 3 nunca produzem uma correspondência (as sobreposições da camada 3 são calculadas e ficam abaixo do limiar), e só os testes unitários exercitam uma correspondência por essas camadas. Os próprios fixtures foram escritos deliberadamente mais fracos para RED e deliberadamente alinhados com a lista esperada para GREEN. Nesses fixtures, o matcher pontua RED com 4% e GREEN com 80% em média — uma afirmação sobre como os fixtures foram escritos, não sobre o ASP.

A propriedade útil que a execução em modo mock de fato estabelece é a **discriminação**: pontuar cada fixture GREEN contra a lista esperada de uma tarefa *diferente* derruba a média de 80% para 0,5% (maior célula fora da tarefa: 10%; Figura 2). Isso é uma evidência mais fraca do que parece: o único acerto fora da tarefa é o nome genérico de uma palavra, “Alerting”, e as demais células são zero porque a camada 1 exige que um nome esperado inteiro apareça literalmente. O matcher também é mais estrito que os arquivos de avaliação, que dizem a um avaliador humano que aceite paráfrases. A revocação é um assunto à parte, e os fixtures mostram um falso negativo: o fixture RED da E2 menciona um limite de taxa, novas tentativas e logging e ainda assim pontua 0/10: sua melhor sobreposição de janela com o ponto esperado “Rate-limit backoff” (seu nome e sua descrição) é um Jaccard de 0,11, abaixo do limiar de 0,30, e o matcher não faz stemming (“logging” não casa com “logs”). Paráfrases com baixa sobreposição lexical são perdidas. A repontuação também expôs um defeito, corrigido na v2.0: a janela deslizante pulava os últimos tokens de um texto cujo comprimento não caía no passo; repontuar todos os fixtures com a correção não altera nenhum valor de cobertura, embora a melhor sobreposição de três pontos sem correspondência suba ligeiramente (E2 RED “Observability / logs” de 0,00 para 0,02), ainda muito

abaixo do limiar. A camada 3 tem também um teto aritmético: o Jaccard divide pela união, e uma janela de 40 tokens dos fixtures contém de 27 a 40 palavras de conteúdo distintas (mediana 36); em qualquer janela com 34 ou mais, um ponto esperado com dez ou menos tokens distintos não consegue alcançar 0,30 mesmo que o plano repita todos eles, e 19 dos 50 pontos esperados são curtos assim. O protocolo em modo real, portanto, muda a camada 3 antes do depósito (PREREGISTRATION.md). O matcher ainda não foi aplicado a nenhuma transcrição real. O protocolo em modo real, com listas esperadas reescritas às cegas por engenheiros independentes, está especificado no PREREGISTRATION.md do repositório e não foi executado. Sua regra de decisão é fixada de antemão: com oito réplicas por tarefa e condição (80 execuções), uma execução é rotulada como incompleta, sem veredito, se o orçamento interromper a coleta antes de cada tarefa chegar a oito réplicas por condição; caso contrário, um benefício só é relatado se a cobertura média da condição assistida superar a da não assistida nas cinco tarefas e um teste de permutação unilateral, estratificado por tarefa, sobre a diferença média por tarefa (10 000 reatribuições de rótulos, semente asp-bench-v1) der $p < 0{,}05$; nenhum benefício mensurável se a condição assistida não superar a não assistida ($\text{GREEN} - \text{RED} \leq 0$, de modo que o empate conta) em três ou mais tarefas; inconclusivo nos demais casos. A cobertura será relatada por camada do matcher, já que a camada 1 premia um plano que reutiliza literalmente os nomes esperados.

Figura 2

Verificação do instrumento: cada fixture GREEN construído pelo autor, pontuado contra a lista esperada de cada tarefa. Não é um efeito do ASP



Qual componente seria o mais importante, se o ASP tiver efeito, é discutido como hipótese na Seção 6.

5.4. O achado do código de saída do claude -p

Durante uma bateria de quatro testes contra o executor não interativo claude -p, observamos que o binário devolve o código de saída 0 mesmo quando o agente recusa com educação o objetivo solicitado — o harness relata um término bem-sucedido da conversa apesar de o agente recusar

semanticamente a tarefa. O código de saída reflete o sucesso em nível de processo do binário da CLI, não o sucesso semântico da tarefa do agente. As Iron Laws 11 e 12 exigem --output-format json ou stream-json e a leitura de is_error, terminal_reason e stop_reason. Esses campos separam um corte pelo limite de turnos e uma recusa de classificador de um turno concluído, mas não uma recusa gerada pelo modelo de uma entrega; o runner, por isso, também sinaliza respostas cujo texto casa com um padrão de recusa, uma heurística que o artigo complementar testa.

Esse achado é observacionalmente significativo para qualquer cadeia de ferramentas que envolva o claude -p num pipeline de shell. Como a observação veio de um único cenário de recusa em nossa própria bateria, nós o **caracterizamos de forma independente** no artigo complementar Flores (2026c), que relata uma medição com N=50 em quatro categorias de prompt feito para provocar recusa (impossível em um turno, capacidade, segurança, ambiguidade) no Claude Code 2.1.200 em julho de 2026. O modelo da sessão foi o Fable 5; o Opus 4.8 respondeu 16 dos 50 prompts, incluindo todos os 12 prompts de segurança — compatível com uma troca automática de modelo (fallback), embora os envelopes registrem qual modelo respondeu, não por quê. Vinte e quatro tentativas saíram com 0, todas com stop_reason: end_turn. O artigo complementar conta as recusas entre elas de quatro formas: 20 (40%, IC de Wilson 95% 28–54%) sob o rótulo não cego do autor; 19 (38%) sob a maioria de um codebook, escrito depois de o autor ter lido as respostas e aplicado por cinco avaliadores LLM, quatro dos quais devolveram rótulos idênticos, todos da linhagem do modelo sob teste, de modo que a maioria é uma checagem do codebook, não uma estimativa de concordância entre avaliadores; 12 (24%) contando apenas as recusas de segurança e de comando destrutivo e excluindo as oito recusas fundamentadas de tarefas impossíveis em um turno; e 9 (18%) sob o detector de frases travado só em inglês, que não pega as respostas escritas em português. Um retrato anterior (maio de 2026, Claude Code 2.1.143, Opus 4.7) leu 60% sob o

mesmo detector, em outro modelo e outro idioma de resposta. Um segundo avaliador humano e um terceiro retrato sob um pré-registro redigido são os desdobramentos em aberto.

6. Hipóteses de Projeto

Destacamos cinco escolhas de projeto que acreditamos serem estruturais (*load-bearing*). As quatro primeiras são hipóteses sobre o ASP que nenhum dado aqui mede, cada uma enunciada com a observação que a motivou; a quinta se apoia na medição do artigo complementar.

Hipótese: uma taxonomia forçada levanta mais do que um pre-mortem livre. Nos rascunhos do autor, o pre-mortem livre se agrupava em categorias de risco óbvias (dados, integração) e subpesava as menos salientes (tempo/fuso horário, observabilidade, excelência operacional, deriva de contrato). Nenhuma execução testou o pre-mortem livre como condição (RED não tinha nenhum framework), de modo que isso se apoia na leitura do autor sobre rascunhos. O prompt livre ("imagine que a tarefa falhou em 30 dias; liste mais de 10 causas") seria então necessário, mas não suficiente, e o interrogatório dos 14 modos do MAST levantaria categorias de que a passagem livre se afasta por agrupamento.

Hipótese: o isolamento de prompt do validador importa. Esperamos que validadores que compartilham contexto com os planejadores reproduzam a confiança do planejador sobre o conteúdo compartilhado. A literatura convergente sobre os limites da autocorreção intrínseca, a falha de localização de erros e o viés de autoaprimoramento dos juízes aponta para isso; não o testamos dentro do ASP.

Hipótese: passagens baseadas em arquivo sobrevivem melhor à pressão de contexto do que as baseadas em transcrição. O ASP grava em disco o charter, o plano macro, o registro de entregáveis e o goal spec antes de criar o executor. O filho recebe apenas o texto do Goal Spec na

entrada padrão e devolve um resultado JSON; o pai reconstrói o estado a partir desse resultado e dos arquivos em disco, de modo que nenhum dos lados depende de uma transcrição que pode ter sido comprimida ou truncada. Não comparamos as duas passagens.

Hipótese: restrições enunciadas (Iron Laws) tornam uma skill mais difícil de pular. Em nossa experiência, skills que se leem como "diretrizes" muitas vezes são puladas sob pressão de contexto. Skills que emitem uma lista de inegociáveis, com templates cujos campos devem ser preenchidos antes da fase seguinte, são, em nosso uso, mais difíceis de pular. O cumprimento não foi medido.

Medido em outro lugar: tratar os códigos de saída do claude -p como sinais de sucesso semântico é uma armadilha silenciosa. Esse é o achado relatado em Flores (2026c); mencionamo-lo aqui porque a implicação de projeto (Iron Laws 11 e 12) foi material para a confiabilidade do sistema.

7. Ameaças à Validade

Tratamos as ameaças de forma explícita nas quatro categorias padrão dos relatos empíricos de engenharia de software.

Validade de construto. O mesmo pesquisador que projetou o protocolo escreveu as listas de shadow points esperados *e* adjudicou manualmente a cobertura. Mesmo com uma rubrica escrita, essa é a maior ameaça isolada à interpretabilidade da observação. O adjudicador determinístico que distribuímos mitigaria isso uma vez aplicado a transcrições reais e preservadas; nesta versão ele só foi executado em fixtures construídos (§5.3), e não aborda a questão anterior de saber se as próprias listas esperadas são enviesadas em direção ao que o ASP levanta. **A reescrita às cegas por revisores independentes é o desdobramento mais forte.**

Validade interna. Com uma execução por célula, GREEN em cinco tarefas e RED em três, a avaliação no máximo sugere que o protocolo pode mudar as transcrições nesses casos; ela *não* basta para estimar um tamanho de efeito, estabelecer causalidade ou sustentar inferência estatística, e nenhum teste estatístico é relatado. Nenhuma ablação formal dos componentes foi executada (a atribuição qualitativa de caráter estrutural à lista de verificação do MAST na §6 é uma hipótese). O limiar de aceitação de 80% é uma escolha do autor e não deriva da literatura anterior.

Validade externa. Um único modelo subjacente (Claude Opus 4.7, presumido a partir da sessão; não registrado por execução), um único ambiente (a estação de trabalho macOS do autor e a versão do Claude Code de maio de 2026, não registrada por execução), cinco tarefas de engenharia tiradas da experiência de domínio do autor. Não fazemos nenhuma alegação de generalização além dessas condições.

Confiabilidade. Todos os enunciados INPUT das avaliações, as listas de shadow points esperados, os fixtures das execuções mock e o código do adjudicador determinístico estão versionados no repositório complementar; as transcrições das execuções de 2026-05-17 não foram preservadas, de modo que as observações da §5.2 não podem ser reverificadas. O pacote de benchmark constrói um manifesto encadeado criptograficamente por execução, de modo que qualquer byte editado sob um diretório de execução invalida o hash do manifesto.

8. Limitações e Trabalhos Futuros

As limitações deste relato de experiência correspondem uma a uma às ameaças acima. Os desdobramentos de maior prioridade, em ordem, são:

1. **Reescrita às cegas** das listas de shadow points esperados por 2 ou mais revisores independentes sem exposição ao ASP, seguida de uma execução em modo real cujas transcrições sejam preservadas e seladas por hash, adjudicada às cegas. Calcular o κ de Cohen para a concordância entre adjudicadores; remedir a cobertura.
2. **Ablação formal** da hipótese do que é estrutural (*load-bearing*): lista de verificação do MAST ligada/desligada (testando a hipótese da §6), isolamento de prompt do validador ligado/desligado, admissão da Fase 0a ligada/desligada. Ainda não existe protocolo de ablação; o Protocolo 2 do PREREGISTRATION.md cobre apenas RED contra GREEN, com 8 réplicas por célula e orçamento fixo, e não uma análise de poder.
3. **Um segundo avaliador humano e um terceiro retrato do claude -p** para o achado complementar (Flores, 2026c, §8), este último sob o Protocolo 1 do PREREGISTRATION.md.
4. **Avaliação entre modelos.** Reexecutar no Claude Sonnet, no GPT-5, no Gemini 2.5 e num modelo de pesos abertos.
5. **Comparação direta com o PreFlect.** Executar o ASP e uma reimplementação do PreFlect no mesmo conjunto de tarefas.
6. **Adjudicador baseado em embeddings** como uma v0.2 do benchmark, com modelo sentence-transformer fixado, para elevar o piso de revocação em paráfrases lexicalmente disjuntas.

7. **Replicação em benchmark padronizado.** O conjunto de cinco tarefas não é um benchmark público de agentes; replicar a direção no SWE-bench (Jimenez et al., 2024) ou num conjunto padronizado comparável testaria se ela sobrevive fora das tarefas escolhidas pelo autor.
8. **Revisão das traduções**: a revisão, pelo autor, da tradução para o português e, por falantes nativos, das traduções da documentação para espanhol, italiano e hebraico.
9. **Alvo de submissão.** Venues de workshop — a trilha de agentes de LLM do NeurIPS, o workshop de agentes do ICLR, a trilha de agentes da AAAI — são o encaixe adequado para a escala empírica atual. A submissão à trilha principal exige os desdobramentos acima.

9. Conclusão

Descrevemos o ASP, uma skill operacional de pre-mortem para agentes de código com LLM, que integra o achado de retrospecção prospectiva de Mitchell, Russo e Pennington (1989), a operacionalização do pre-mortem por Klein (2007) e a taxonomia de falhas de 14 modos MAST de Cemri et al. (2025) como disciplina de tempo de execução sob um contrato de doze Iron Laws. A skill é distribuída em código aberto por três caminhos de instalação. Nas três tarefas executadas nas duas condições, o autor observou, sem cegamento e sobre transcrições que não foram preservadas, que os planos assistidos cobriram mais dos modos de falha esperados; nenhum efeito foi medido. O adjudicador determinístico distribuído com a skill foi exercitado apenas em fixtures construídos, nos quais separa nomes esperados inteiros mas não tem revocação medida; uma execução em modo real com listas escritas de forma independente, redigida para pré-registro, é o próximo passo decisivo.

Observamos, além disso, que o claude -p devolve o código de saída 0 mesmo quando o agente recusa com educação o objetivo solicitado. O fornecedor documenta o comportamento para a sua API; o artigo complementar (Flores, 2026c) acrescenta uma contagem na fronteira da CLI.

O desdobramento mais forte é a readjudicação às cegas. O trabalho complementar mais útil é um segundo avaliador humano e um terceiro retrato pré-registrado da medição do claude -p (Flores, 2026c).

Agradecimentos

Este relato de experiência foi preparado em explícita colaboração com humano no ciclo com o Claude (Anthropic, Opus 4.7), usado como parceiro de redação, verificação e revisão. As decisões de metodologia, o desenho empírico e a finalização foram feitos pelo autor humano. O modelo de colaboração está documentado nas Partes 5–6 do docs/ARCHITECTURE.md do repositório complementar.

O autor agradece aos mantenedores do framework de skills obra/superpowers, aos contribuidores do padrão aberto agentskills.io e à equipe do Claude Code da Anthropic pela primitiva /goal, cujo comportamento empírico motivou as Iron Laws 11 e 12.

Referências (APA, 7.ª edição)

Anthropic. (2026a). *Claude Code* [Software documentation]. https://code.claude.com

Anthropic. (2026b). *Agent Skills best practices* [Software documentation]. https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices

Avizienis, A., Laprie, J.-C., Randell, B., & Landwehr, C. (2004). Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing*, *1*(1), 11–33. https://doi.org/10.1109/TDSC.2004.2

Barke, S., Goyal, A., Khare, A., Singh, A., Nath, S., & Bansal, C. (2026). *AgentRx: Diagnosing AI agent failures from execution trajectories*. arXiv:2602.02475. https://arxiv.org/abs/2602.02475

Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., & Stoica, I. (2025). Why do multi-agent LLM systems fail? *Advances in Neural Information Processing Systems, 38*, 135676–135729. https://doi.org/10.52202/085713-4082

Deshpande, D., Gangal, V., Mehta, H., Krishnan, J., Kannappan, A., & Qian, R. (2025). *TRAIL: Trace reasoning and agentic issue localization*. arXiv:2505.08638. https://arxiv.org/abs/2505.08638

Erdogan, L. E., Lee, N., Kim, S., Moon, S., Furuta, H., Anumanchipalli, G., Keutzer, K., & Gholami, A. (2025). *Plan-and-Act: Improving planning of agents for long-horizon tasks*. ICML 2025. https://arxiv.org/abs/2503.09572

Flores, C. U. (2026a). *ASP — Anticipating Shadow Points: Claude Code plugin and replication package* (Version 2.0.0) [Computer software].

https://github.com/ulissesflores/anticipating-shadow-points. https://doi.org/10.5281/zenodo.20276631

Flores, C. U. (2026c). *Graceful refusals as unsignalled failures: Characterising claude -p exit-code semantics on agent declines* (Version 2.2) [Technical note]. In Flores (2026a). https://doi.org/10.5281/zenodo.20276631

Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., & Zhou, D. (2024). *Large language models cannot self-correct reasoning yet*. ICLR 2024. https://arxiv.org/abs/2310.01798

IEEE. (2017). *IEEE standard for system, software, and hardware verification and validation* (IEEE Std 1012-2016). https://doi.org/10.1109/IEEESTD.2017.8055462

International Electrotechnical Commission. (2016). *Hazard and operability studies (HAZOP studies) — Application guide* (IEC 61882:2016, Edition 2.0). https://webstore.iec.ch/en/publication/24321

International Electrotechnical Commission. (2018). *Failure modes and effects analysis (FMEA and FMECA)* (IEC 60812:2018). https://webstore.iec.ch/en/publication/26359

International Electrotechnical Commission. (2019). *Risk management — Risk assessment techniques* (IEC 31010:2019, Edition 2.0). https://webstore.iec.ch/en/publication/59809

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). *SWE-bench: Can language models resolve real-world GitHub issues?* ICLR 2024. https://arxiv.org/abs/2310.06770

Kambhampati, S., Valmeekam, K., Guan, L., Verma, M., Stechly, K., Bhambri, S., Saldyt, L., & Murthy, A. (2024). *LLMs can't plan, but can help planning in LLM-Modulo frameworks*. ICML 2024 (position paper). https://arxiv.org/abs/2402.01817

Kamoi, R., Zhang, Y., Zhang, N., Han, J., & Zhang, R. (2024). When can LLMs actually correct their own mistakes? A critical survey of self-correction of LLMs. *Transactions of the Association for Computational Linguistics*. https://arxiv.org/abs/2406.01297

Klein, G. (2007). Performing a project premortem. *Harvard Business Review*, *85*(9), 18–19.

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A., & Clark, P. (2023). *Self-Refine: Iterative refinement with self-feedback*. NeurIPS 2023. https://arxiv.org/abs/2303.17651

Meyer, B. (1992). Applying "design by contract." *Computer*, *25*(10), 40–51. https://doi.org/10.1109/2.161279

Mitchell, D. J., Russo, J. E., & Pennington, N. (1989). Back to the future: Temporal perspective in the explanation of events. *Journal of Behavioral Decision Making*, *2*(1), 25–38. https://doi.org/10.1002/bdm.3960020103

Popper, K. R. (2002). *The logic of scientific discovery*. Routledge. (Original work published 1959)

Shingo, S. (1986). *Zero quality control: Source inspection and the poka-yoke system* (A. P. Dillon, Trans.). Productivity Press. (Original work published 1985)

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). *Reflexion: Language agents with verbal reinforcement learning*. NeurIPS 2023. https://arxiv.org/abs/2303.11366

Tyen, G., Mansoor, H., Cărbune, V., Chen, P., & Mak, T. (2024). LLMs cannot find reasoning errors, but can correct them given the error location. In *Findings of the Association for Computational Linguistics: ACL 2024*. https://arxiv.org/abs/2311.08516

U.S. Department of Defense. (1980). *Procedures for performing a failure mode, effects and criticality analysis* (MIL-STD-1629A). https://everyspec.com/MIL-STD/MIL-STD-1600-1699/MIL_STD_1629A_1556/

Wang, H., Cao, Y., Lin, L., & Chen, J. (2026). *PreFlect: From retrospective to prospective reflection in large language model agents*. arXiv. https://arxiv.org/abs/2602.07187

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., & Wang, C. (2023). *AutoGen: Enabling next-gen LLM applications via multi-agent conversation*. arXiv. https://arxiv.org/abs/2308.08155

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *ReAct: Synergizing reasoning and acting in language models*. ICLR 2023. https://arxiv.org/abs/2210.03629

Yuan, D., Luo, Y., Zhuang, X., Rodrigues, G. R., Zhao, X., Zhang, Y., Jain, P. U., & Stumm, M. (2014). Simple testing can prevent most critical failures: An analysis of production failures in distributed data-intensive systems. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI '14)* (pp. 249–265). USENIX Association. https://www.usenix.org/conference/osdi14/technical-sessions/presentation/yuan

Zhang, G., Wang, J., Chen, J., Zhou, W., Wang, K., & Yan, S. (2025). *AgenTracer: Who is inducing failure in the LLM agentic systems?* arXiv:2509.03312. https://arxiv.org/abs/2509.03312

Zhang, J., Peng, X., Chen, Q., Li, Y., Hayashi, H., & Wu, C.-S. (2026). *Prospective hindsight: Self-calibrating reinforcement learning via prediction-reality gaps*. arXiv:2610.02740. https://arxiv.org/abs/2610.02740

Zhang, S., Yin, M., Zhang, J., Liu, J., Han, Z., Zhang, J., Li, B., Wang, C., Wang, H., Chen, Y., & Wu, Q. (2025). *Which agent causes task failures and when? On automated failure attribution of LLM multi-agent systems*. ICML 2025 Spotlight. https://arxiv.org/abs/2505.00212

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS 2023 D&B. https://arxiv.org/abs/2306.05685

Zhu, K., Liu, Z., Li, B., Tian, M., Yang, Y., Zhang, J., Han, P., Xie, Q., Cui, F., Zhang, W., Ma, X., Yu, X., Ramesh, G., Wu, J., Liu, Z., Lu, P., Zou, J., & You, J. (2025). *Where LLM agents fail and how they can learn from failures*. arXiv. https://arxiv.org/abs/2509.25370

Apêndice A — Situação da verificação das citações

Toda afirmação do corpo cita uma fonte primária quando ela existe. Os parágrafos abaixo registram quando cada grupo de referências foi conferido.

As referências presentes na versão 1.0 foram verificadas por consulta direta à URL em 2026-05-17; as referências acrescentadas na revisão de julho de 2026 (Flores, 2026c; as entradas da linhagem clássica da Seção 2.5) foram conferidas quando acrescentadas; e as entradas acrescentadas em novembro de 2026 (Barke et al., 2026; Deshpande et al., 2025; Kambhampati et al., 2024; G. Zhang et al., 2025; J. Zhang et al., 2026) foram conferidas contra seus registros no arXiv, e Cemri et al. (2025) contra o seu registro nos anais do NeurIPS. Os nomes dos 14 modos do MAST são citados literalmente de Cemri et al. (2025) e listados no arquivo complementar da skill skills/anticipating-shadow-points/mast-checklist.md. Mitchell et al. (1989) é o estudo primário da retrospecção prospectiva; a cifra de ≈30% é o resumo que Klein (2007) faz dele, que não conseguimos recalcular a partir do artigo de 1989, e Klein (2007) é também a receita operacional.

As nove referências introduzidas na §2.5 (Avizienis et al., 2004; International Electrotechnical Commission [IEC 60812:2018], 2018; International Electrotechnical Commission [IEC 61882:2016], 2016, conferida no catálogo da IEC em 2026-11-07; IEEE [Std 1012-2016], 2017; Meyer, 1992; Popper, 1959/2002; Shingo, 1986; U.S. Department of Defense [MIL-STD-1629A], 1980; Yuan et al., 2014) foram verificadas contra a sua fonte primária — DOI, ISBN ou entrada de catálogo de norma — em 2026-07-08.

Apêndice B — Conjunto de avaliações e reprodutibilidade

As cinco avaliações de engenharia estão versionadas no repositório complementar:

- skills/anticipating-shadow-points/evals/01-supabase-migration.md (E1)
- skills/anticipating-shadow-points/evals/02-edge-function-deploy.md (E2)
- skills/anticipating-shadow-points/evals/03-refactor-shared-util.md (E3)
- skills/anticipating-shadow-points/evals/04-rls-policy-change.md (E4)
- skills/anticipating-shadow-points/evals/05-cron-skill-conflict.md (E5)

Cada arquivo contém o enunciado INPUT, a lista de shadow points EXPECTED e o limiar de ACCEPTANCE, todos escritos pelo projetista do protocolo. O pacote de benchmark em benchmark/ fornece:

- um adjudicador determinístico de 4 camadas (exato / sinônimo / Jaccard / sem correspondência);
- proveniência criptográfica por execução (ambiente, código, entradas, saídas e adjudicação com hash num encadeamento ao estilo Merkle);
- um subcomando verify que percorre de novo um diretório de execução e relata qualquer deriva em nível de byte;
- 31 testes unitários que cobrem o adjudicador, os analisadores de avaliações e de respostas, o manifesto e o cálculo de hash, incluindo o contrato de determinismo.

Reproduzindo os números calculados das Seções 5.3–5.4 (sem chamadas de modelo). A partir de um clone do repositório, python run_all.py repontua os fixtures construídos pelo autor da execução mock do benchmark com o adjudicador distribuído, recalcula a matriz de

discriminação, regenera a Figura 2, verifica cada número (tests/test_paper_numbers.py, que também verifica que a execução está rotulada como mock) e confere um único hash em cadeia SHA-256 sobre código, testes, entradas congeladas, dados e resultados (output/hash-chain.md). As observações do autor na Seção 5.2 não podem ser reproduzidas: suas transcrições não foram preservadas. Esta versão substitui a v1.0.3 (https://doi.org/10.5281/zenodo.20277015), que apresentava as pontuações do adjudicador sobre esses fixtures como uma leitura de transcrições de modelos.

Coletando evidência real (consome chamadas de modelo). Instale o pacote de benchmark e execute asp-benchmark run --mode real --model <model>, seguido de asp-benchmark verify <run-id>. Isso não foi feito para nenhum número publicado, e o comando de hoje ainda não é o Protocolo 2: ele envia um template de um único turno em vez da skill distribuída, executa uma réplica por tarefa numa ordem fixa e não passa flags de ambiente limpo. O PREREGISTRATION.md lista as mudanças de que o runner e o matcher precisam antes de qualquer despacho. O protocolo é especificado independentemente da implementação em Python em benchmark/PROTOCOL.md, de modo que pode ser reimplementado em outra linguagem.

Sobre o Autor

Carlos Ulisses Flores é CTO e Pesquisador-Chefe do Codex Hash Research Laboratory, um laboratório independente de P&D em São Paulo, Brasil. Constrói sistemas de produção em que a falha silenciosa é cara — historicamente em trading algorítmico, IoT embarcada e criptografia de borda, e atualmente em ferramentas de IA agêntica. A skill ASP descrita neste whitepaper nasceu dessa última linha de trabalho; o achado empírico relatado no whitepaper complementar nasceu do uso do executor claude -p da Anthropic como núcleo de execução autônoma e de vê-lo devolver o código de saída 0 diante de uma recusa educada do agente.

Escreve em português, inglês, espanhol e italiano.

Identificadores · ORCID: 0000-0002-6034-7765 · Lattes: 6905246706890561

Contato · c.ulisses@gmail.com · https://ulissesflores.com